

بازی زندگی کانوی و مورچه لنگتون

چکیده

در این پروژه، دو اتوماتای سلولی مشهور یعنی بازی زندگی کانوی (Conway's Game of Life) و مورچه لنگتون (Langton's Ant) پیاده‌سازی شده‌اند. این پروژه نشان می‌دهد که چگونه قوانین محلی بسیار ساده می‌توانند به رفتارهای سراسری پیچیده و حتی جامعیت تورینگ (Turing Completeness) منجر شوند. در بخش اول، شبیه‌ساز بازی زندگی با قابلیت‌های پیشرفته از جمله پشتیبانی از شرایط مرزی مختلف، بهینه‌سازی محاسبات با استفاده از کانولوشن و بارگذاری الگوها از فایل‌های مختلف پیاده‌سازی شده است. در بخش دوم، مورچه لنگتون با پشتیبانی از حالت‌های چندرنگی پیاده‌سازی شده و رفتار آشوبناک و نظم ظهور یافته آن مورد بررسی قرار گرفته است. در بخش سوم، با طراحی گیت‌های منطقی AND و NOT با استفاده از برخورد گالیدرها، جامعیت تورینگ بازی زندگی به اثبات رسیده است.

مقدمه

اتوماتای سلولی (Cellular Automata) مدل‌های ریاضی گسسته‌ای هستند که از شبکه‌ای از سلول‌ها تشکیل شده‌اند. هر سلول در یکی از حالت‌های متناهی قرار داشته و حالت بعدی آن بر اساس حالت فعلی خود و همسایگانش تعیین می‌شود. یکی از شگفت‌انگیزترین ویژگی‌های این سیستم‌ها، پدیده «نوظهوری» (Emergence) است؛ جایی که از قوانین محلی بسیار ساده، رفتارهای سراسری فوق‌العاده پیچیده و حتی جامعیت تورینگ پدیدار می‌شود.

در این پروژه با دو اتوماتای سلولی معروف کار می‌کنیم:

بازی زندگی کانوی: یک اتوماتای سلولی دوبعدی با قوانین ساده که در سال ۱۹۷۰ توسط جان کانوی معرفی شد. این بازی با وجود قوانین بسیار ساده، قادر به تولید الگوهای پیچیده و حتی انجام محاسبات عمومی است.

مورچه لنگتون: یک اتوماتای سلولی دیگر که توسط کریس لنگتون در سال ۱۹۸۶ معرفی شد. این سیستم از یک مورچه تشکیل شده که روی صفحه‌ای از سلول‌های رنگی حرکت می‌کند و با وجود قوانین بسیار ساده، رفتارهای آشوبناک و منظم جالبی از خود نشان می‌دهد.

بخش اول: بازی زندگی کانوی (Conway's Game of Life)

مبانی نظری

بازی زندگی کانوی بر روی یک شبکه دوبعدی با همسایگی ۸-اتصال مور (Moore Neighborhood) اجرا می‌شود. هر سلول می‌تواند در یکی از دو حالت زنده (۱) یا مرده (۰) باشد. قوانین انتقال به صورت زیر تعریف می‌شوند:

۱. زنده ماندن (Survival): یک سلول زنده با ۲ یا ۳ همسایه زنده، زنده می‌ماند

۲. مرگ بر اثر کم جمعیتی (Underpopulation): یک سلول زنده با کمتر از ۲ همسایه زنده، می‌میرد

۳. مرگ بر اثر ازدحام (Overpopulation): یک سلول زنده با بیش از ۳ همسایه زنده، می‌میرد

۴. تولیدمثل (Reproduction): یک سلول مرده با دقیقاً ۳ همسایه زنده، زنده می‌شود

این قوانین ساده، قابلیت تولید الگوهای بسیار پیچیده را دارند. برخی از الگوهای معروف عبارتند از:

نوسانگر (Oscillator): الگویی که پس از چند نسل به حالت اولیه بازمی‌گردد (مانند Blinker با دوره تناوب ۲)

فضاپیما (Spaceship): الگویی که در صفحه حرکت می‌کند (مانند Glider با سرعت $c/4$)

تفنگ (Gun): الگویی که به طور مداوم فضاپیما تولید می‌کند (مانند Gosper Glider Gun)

پیاده‌سازی کلاس GameOfLife

کلاس GameOfLife در فایل conway.py پیاده‌سازی شده است. این کلاس شامل متدهای زیر می‌باشد:

متد سازنده (`__init__`): این متد، بازی زندگی را با اندازه شبکه، نوع شرایط مرزی (محدود یا حلقوی) و حالت اجرای سریع یا کند مقداردی اولیه می‌کند. شبکه با استفاده از آرایه‌های NumPy ذخیره می‌شود.

متد evolve : این متد، یک نسل از بازی را با استفاده از روش سلول به سلول (در حالت کند) یا با استفاده از کانولوشن (در حالت سریع) محاسبه می‌کند.

متد update_grid_fast : این متد، نسل بعدی را با استفاده از تابع `scipy.signal.convolve2d` محاسبه می‌کند که سرعت محاسبه را به طور قابل توجهی افزایش می‌دهد.

متدهای درج الگو: متدهایی مانند `insertBlinker` ، `insertGlider` و `insertGliderGun` برای قرار دادن الگوهای اولیه در شبکه استفاده می‌شوند.

متد insertFromFile : این متد، الگوها را از فایل‌های با فرمت Plaintext یا RLE بارگذاری می‌کند.

وظیفه ۱-الف: پیاده‌سازی قوانین اصلی

در این بخش، قوانین اصلی بازی زندگی در متد `evolve` پیاده‌سازی شده است. این پیاده‌سازی با استفاده از حلقه‌های تودرتو بر روی سطرها و ستون‌ها، تعداد همسایه‌های هر سلول را محاسبه کرده و قوانین چهارگانه را اعمال می‌کند.

شرایط مرزی:

در حالت محدود (`finite=True`) ، سلول‌های لبه، همسایگانی خارج از شبکه ندارند

در حالت حلقوی (`finite=False`) ، لبه‌های شبکه به یکدیگر متصل شده و شبکه به صورت یک چنبره (Torus) در نظر گرفته می‌شود

تست Blinker : برای تأیید صحت پیاده‌سازی، الگوی Blinker (یک خط سه‌تایی) در مرکز شبکه قرار داده شده است. این الگو باید با دوره تناوب ۲ نوسان کند:

نسل ۰: سه سلول زنده به صورت افقی

نسل ۱: سه سلول زنده به صورت عمودی

نسل ۲: بازگشت به حالت افقی

نتایج تست نشان می‌دهد که پیاده‌سازی به درستی عمل کرده و Blinker با دوره تناوب ۲ نوسان می‌کند.

وظیفه ۱-ب: شبیه‌سازی گالیدر (Glider)

گالیدر یکی از معروف‌ترین الگوهای بازی زندگی است که به صورت مورب در صفحه حرکت می‌کند. این الگو با اندازه 3×3 به صورت زیر تعریف می‌شود:

0.

..0

000

گالیدر با سرعت $c/4$ (یک سلول در هر ۴ نسل) حرکت می‌کند و شکل خود را در طول حرکت حفظ می‌کند. این ویژگی آن را به یک کاندیدای ایده‌آل برای انتقال سیگنال در محاسبات تبدیل می‌کند.

کاربردهای گالیدر در محاسبات:

۱. انتقال اطلاعات: گالیدرها می‌توانند به عنوان حامل‌های اطلاعات در سراسر جهان بازی زندگی حرکت کنند. وجود یا عدم وجود یک گالیدر در یک زمان و مکان خاص می‌تواند نشان‌دهنده بیت‌های ۱ یا ۰ باشد.

۲. ساختارهای محاسباتی: با برخورد دادن گالیدرها با یکدیگر یا با سایر الگوها، می‌توان عملیات منطقی را پیاده‌سازی کرد. برای مثال، برخورد دو گالیدر می‌تواند منجر به تولید الگوهای جدید یا نابودی متقابل شود.

۳. بلوک‌های ساختمانی: گالیدرها به عنوان بلوک‌های ساختمانی اصلی در طراحی مدارهای پیچیده‌تر مانند جمع‌کننده‌ها، حافظه‌ها و حتی پردازنده‌های کامل استفاده می‌شوند.

وظیفه ۱-پ: اشکال‌زدایی تفنگ گالیدر گاسپر

تفنگ گالیدر گاسپر (Gosper Glider Gun) یک الگوی نوسانی است که به طور مداوم جریانی از گالیدرها تولید می‌کند. این الگو اولین تفنگ گالیدر کشف شده است و نقش مهمی در اثبات جامعیت تورینگ بازی زندگی دارد.

مشکل موجود: متد `insertGliderGun` در کد اولیه دارای یک خطای جزئی در مختصات بلوک چپ بود که باعث می‌شد تفنگ به درستی کار نکند.

راه‌حل: با تحلیل مختصات و مقایسه با الگوی استاندارد تفنگ گالیدر گاسپر، خطا شناسایی و اصلاح شد. پس از اصلاح، تفنگ به درستی کار کرده و جریانی نامتناهی از گالیدرها تولید می‌کند.

وظیفه ۱-ت: پیاده‌سازی پارسر فایل‌های الگو

برای بارگذاری الگوهای پیچیده‌تر از فایل‌ها، تابع `parse_pattern` پیاده‌سازی شده است که از دو فرمت پشتیبانی می‌کند:

فرمت: `(.cells)` Plaintext

ساده‌ترین فرمت برای نمایش الگوها

هر سطر نشان‌دهنده یک ردیف از شبکه است

کاراکتر 0 نشان‌دهنده سلول زنده و 1 یا b نشان‌دهنده سلول مرده است

خطوط شروع‌شونده با ! به عنوان کامنت در نظر گرفته می‌شوند

فرمت: RLE (Run-Length Encoded)

فرمت فشرده‌تری برای الگوهای بزرگ

از اعداد برای نشان‌دادن تعداد تکرار هر نماد استفاده می‌کند

0 نشان‌دهنده سلول زنده، 1 نشان‌دهنده سلول مرده

\$ نشان‌دهنده پایان ردیف و ! نشان‌دهنده پایان الگو است

هدر شامل اطلاعات اندازه الگو می‌باشد: x=width, y=height :

پیاده‌سازی: تابع parse_pattern با تشخیص خودکار فرمت فایل، آن را تجزیه کرده و مختصات سلول‌های زنده را باز می‌گرداند.

وظیفه ۱-ث: بهینه‌سازی محاسبات با کانولوشن

برای شبکه‌های بزرگ ($N > 1024$)، محاسبه سلول به سلول بسیار کند است. برای حل این مشکل، متد

update_grid_fast با استفاده از کانولوشن دو بعدی پیاده‌سازی شده است:

روش کار:

۱. استفاده از تابع `scipy.signal.convolve2d` برای محاسبه تعداد همسایه‌های هر سلول

۲. استفاده از هسته (Kernel) 3×3 با مقدار 1 در همه خانه‌ها به جز مرکز (که 0 است)

۳. اعمال قوانین به صورت برداری با استفاده از عملگرهای منطقی NumPy

مزایا:

سرعت بسیار بالاتر برای شبکه‌های بزرگ (حدود 10 تا 100 برابر سریع‌تر)

استفاده از بهینه‌سازی‌های سطح پایین کتابخانه SciPy

نتایج دقیقاً مشابه روش سلول به سلول

تست عملکرد: مقایسه زمان اجرای روش کند و سریع برای ۵۰ نسل از یک شبکه 256×256 تصادفی نشان می‌دهد که روش سریع به طور قابل توجهی کارآمدتر است.

بخش دوم: مورچه لنگتون (Langton's Ant)

مبانی نظری

مورچه لنگتون یک اتوماتای سلولی دوبعدی است که توسط کریس لنگتون در سال ۱۹۸۶ معرفی شد. این سیستم شامل یک مورچه است که روی شبکه‌ای از سلول‌های رنگی حرکت می‌کند. قوانین اولیه با دو رنگ سفید و سیاه به صورت زیر تعریف می‌شوند:

۱. روی سلول سفید: رنگ سلول را به سیاه تغییر می‌دهد، ۹۰ درجه به راست می‌چرخد و یک سلول به جلو حرکت می‌کند

۲. روی سلول سیاه: رنگ سلول را به سفید تغییر می‌دهد، ۹۰ درجه به چپ می‌چرخد و یک سلول به جلو حرکت می‌کند

با وجود این قوانین بسیار ساده، مورچه رفتارهای پیچیده‌ای از خود نشان می‌دهد:

رفتار آشوبناک: در هزاران گام اول، مورچه به طور نامنظم حرکت کرده و الگوهای متقارن نامنظم ایجاد می‌کند
بزرگراه: پس از حدود ۱۰,۰۰۰ گام، مورچه ناگهان وارد یک الگوی تکراری ۱۰۴ گامی می‌شود که به صورت مورب به طور نامتناهی پیشروی می‌کند

پیاده‌سازی کلاس LangtonsAnt

کلاس LangtonsAnt در فایل langton.py پیاده‌سازی شده است. این کلاس شامل اجزای زیر می‌باشد:

متد سازنده (`__init__`): این متد، مورچه را با اندازه شبکه، موقعیت شروع، جهت شروع و قوانین انتقال مقداردهی اولیه می‌کند.

مدیریت جهت: چهار جهت اصلی (شمال، شرق، جنوب، غرب) به صورت لیستی از رشته‌ها تعریف شده و حرکت‌های متناظر با آنها در دیکشنری MOVES ذخیره شده است.

متدهای چرخش: متدهای `turn_left` و `turn_right` برای چرخش ۹۰ درجه مورچه در جهت عقربه‌های ساعت یا خلاف آن استفاده می‌شوند.

متد `move_forward`: این متد، مورچه را یک سلول در جهت فعلی حرکت می‌دهد. در صورت خروج از مرز شبکه، موقعیت به صورت حلقوی (با عملگر مد) به سمت مخالف منتقل می‌شود.

متد `step`: این متد، یک گام از حرکت مورچه را اجرا می‌کند:

۱. خواندن رنگ سلول فعلی

۲. تغییر رنگ سلول بر اساس قوانین

۳. چرخش بر اساس قوانین

۴. حرکت به سلول بعدی

وظیفه ۲-الف: پیاده‌سازی پایه مورچه

مورچه لنگتون پایه با دو رنگ سفید و سیاه پیاده‌سازی شده است. قوانین به صورت دیکشنری زیر تعریف می‌شوند:

```
DEFAULT_RULES = {
```

```
    رنگ ۰ (سفید) ← تغییر به رنگ ۱ (سیاه) و چرخش به راست # (1, "R"),
```

```
    رنگ ۱ (سیاه) ← تغییر به رنگ ۰ (سفید) و چرخش به چپ # (0, "L"),
```

```
}
```

وظیفه ۲-ب: شبیه‌سازی و اثبات سناریوها

رفتار آشوبناک: در گام‌های ابتدایی (حدود ۱۰,۰۰۰ گام اول)، مورچه به صورت کاملاً نامنظم حرکت می‌کند. الگوهای ایجاد شده نامتقارن و غیرقابل پیش‌بینی هستند. این رفتار نشان‌دهنده حساسیت به شرایط اولیه و آشوب در سیستم‌های ساده است.

بزرگراه مورچه: پس از حدود ۱۰,۰۰۰ گام، مورچه به طور ناگهانی وارد یک الگوی تکراری می‌شود. این الگو که به «بزرگراه» معروف است، شامل یک حرکت مورب است که به طور نامتناهی ادامه می‌یابد. دوره تناوب این الگو ۱۰۴ گام است و مورچه در هر دوره، ۲ سلول به سمت راست و ۲ سلول به سمت پایین حرکت می‌کند. این گذار از آشوب به نظم، یکی از جذاب‌ترین جنبه‌های مورچه لنگتون است و مفهوم ظهور (Emergence) را به خوبی نشان می‌دهد.

وظیفه ۲-پ: بهبود مورچه با پشتیبانی از چند رنگ

مورچه لنگتون چندرنگی با تعمیم قوانین به بیش از دو رنگ پیاده‌سازی شده است. در این نسخه، هر سلول می‌تواند یکی از چندین رنگ را داشته باشد و قوانین به صورت دیکشنری زیر تعریف می‌شوند:

```
MULTI_COLOR_RULES = {
```

```
    رنگ ۰ ← رنگ ۱ و چرخش به راست # (1, "R"),
```

```
    رنگ ۱ ← رنگ ۲ و چرخش به چپ # (2, "L"),
```

رنگ ۲ ← رنگ ۳ و چرخش به راست # (3, "R"), 2:

رنگ ۳ ← رنگ ۰ و چرخش به چپ # (0, "L"), 3:

}

با تغییر قوانین، الگوهای زیبا و متنوعی ایجاد می‌شوند. برخی از این الگوها شامل تقارن‌های جالب و اشکال هندسی منظم هستند. این قابلیت، قدرت اتوماتای سلولی در تولید الگوهای پیچیده از قوانین ساده را نشان می‌دهد.

بخش سوم: جامعیت تورینگ از طریق گیت‌های منطقی

مبانی نظری

جامعیت تورینگ به معنای توانایی یک سیستم در شبیه‌سازی هر ماشین تورینگ دلخواه است. برای اثبات جامعیت تورینگ بازی زندگی، کافی است نشان دهیم که می‌توان گیت‌های منطقی اصلی (AND) و (NOT) را در این جهان پیاده‌سازی کرد. با ترکیب این گیت‌ها، می‌توان هر مدار دیجیتالی را ساخت.

در بازی زندگی، سیگنال‌ها با استفاده از گالیدرها نمایش داده می‌شوند:

بیت ۱: حضور یک گالیدر در یک زمان و مکان مشخص

بیت ۰: عدم حضور گالیدر

پیاده‌سازی گیت AND

گیت AND با استفاده از برخورد دو گالیدر در زاویه ۹۰ درجه پیاده‌سازی شده است:

ساختار فیزیکی:

گالیدر A از سمت بالا به پایین حرکت می‌کند

گالیدر B از سمت چپ به راست حرکت می‌کند

مسیرها در نقطه‌ای مشخص تقاطع دارند

جدول درستی:

ورودی (۰,۰): هیچ گالیدری شلیک نمی‌شود ← خروجی ۰

ورودی (۱,۰): فقط گالیدر A حرکت می‌کند ← از نقطه تقاطع عبور کرده و محو می‌شود ← خروجی ۰

ورودی (۰,۱): فقط گالیدر B حرکت می‌کند ← از نقطه تقاطع عبور کرده و محو می‌شود ← خروجی ۰

ورودی (۱,۱): هر دو گالیدر شلیک می‌شوند ← در نقطه تقاطع برخورد کرده و بلوک ۲×۲ تشکیل می‌دهند ← خروجی ۱

منطق عملکرد: برخورد دو گالیدر در نقطه تقاطع باعث ایجاد یک الگوی پایدار (بلوک ۲×۲) می‌شود که در منطقه خروجی قابل تشخیص است. این بلوک نشان‌دهنده خروجی ۱ است.

پیاده‌سازی گیت NOT

گیت NOT با استفاده از یک گالیدر کنترل و یک گالیدر ورودی پیاده‌سازی شده است:

ساختار فیزیکی:

گالیدر کنترل همیشه شلیک می‌شود و به سمت منطقه خروجی حرکت می‌کند

گالیدر ورودی در صورت فعال بودن، در مسیر گالیدر کنترل قرار می‌گیرد

جدول درستی:

ورودی ۰: گالیدر کنترل بدون برخورد عبور کرده و به منطقه خروجی می‌رسد ← خروجی ۱

ورودی ۱: گالیدر ورودی با گالیدر کنترل برخورد کرده و هر دو نابود می‌شوند ← خروجی ۰

منطق عملکرد: برخورد دو گالیدر باعث نابودی متقابل (Annihilation) می‌شود. در نتیجه، در حالت ورودی ۱، هیچ گالیدری به منطقه خروجی نمی‌رسد که معادل خروجی ۰ است.

نتایج تست گیت‌ها

تست‌های انجام شده بر روی گیت‌های AND و NOT نتایج زیر را نشان می‌دهند:

گیت: AND

AND (۰,۰) ← خروجی: ۰ ✓

AND (۰,۱) ← خروجی: ۰ ✓

AND (۱,۰) ← خروجی: ۰ ✓

AND (۱,۱) ← خروجی: ۱ ✓

گیت: NOT

NOT (۰) ← خروجی: ۱ ✓

NOT (۱) ← خروجی: ۰ ✓

همه تست‌ها با موفقیت پاس شده‌اند که نشان‌دهنده صحت پیاده‌سازی گیت‌های منطقی است.

دلالت‌های جامعیت تورینگ

با داشتن گیت‌های AND و NOT ، می‌توان:

گیت NAND را ساخت (ترکیب AND و NOT)

با استفاده از گیت NAND ، هر تابع بولی دیگر را پیاده‌سازی کرد

مدارهای ترکیبی (Combinational Circuits) مانند جمع‌کننده‌ها و ضرب‌کننده‌ها را ساخت

مدارهای ترتیبی (Sequential Circuits) مانند فلیپ‌فلاپ‌ها و حافظه‌ها را طراحی کرد

در نهایت، یک کامپیوتر کامل را در بازی زندگی پیاده‌سازی کرد

این اثبات نشان می‌دهد که بازی زندگی کانوی، با وجود قوانین بسیار ساده، به اندازه کامپیوترهای مدرن قدرتمند است و می‌تواند هر الگوریتم قابل‌محاسبه‌ای را اجرا کند.

نتیجه‌گیری

در این پروژه، دو اتوماتای سلولی مشهور یعنی بازی زندگی کانوی و مورچه لنگتون پیاده‌سازی و مورد بررسی قرار گرفتند. نتایج این پروژه نشان می‌دهند که:

۱. قوانین ساده، رفتارهای پیچیده: با وجود قوانین محلی بسیار ساده، اتوماتای سلولی قادر به تولید رفتارهای سراسری فوق‌العاده پیچیده هستند. این پدیده که «نوظهوری» نام دارد، در هر دو سیستم مشاهده شد.

۲. گذار از آشوب به نظم: مورچه لنگتون به خوبی نشان می‌دهد که چگونه یک سیستم ساده می‌تواند از رفتار آشوبناک اولیه به الگوهای منظم و تکراری گذار کند. این گذار پس از حدود ۱۰,۰۰۰ گام رخ می‌دهد و نشان‌دهنده ظهور نظم از آشوب است.

۳. جامعیت تورینگ: با پیاده‌سازی گیت‌های AND و NOT در بازی زندگی، جامعیت تورینگ این اتوماتای سلولی به اثبات رسید. این بدان معناست که بازی زندگی می‌تواند هر محاسبه قابل‌حل توسط کامپیوترهای مدرن را شبیه‌سازی کند.

۴. اهمیت بهینه‌سازی: برای شبکه‌های بزرگ، استفاده از کانولوشن و محاسبات برداری، سرعت شبیه‌سازی را به طور قابل توجهی افزایش می‌دهد و امکان مطالعه الگوهای بزرگ‌تر را فراهم می‌کند.

۵. ابزارهای بصری: پیاده‌سازی نمایشگرهای بصری با استفاده از PyGame ، امکان مشاهده و تحلیل رفتار این سیستم‌ها را در زمان واقعی فراهم می‌کند که برای درک بهتر مفاهیم بسیار مفید است.

در مجموع، این پروژه نشان می‌دهد که چگونه مدل‌های ساده ریاضی می‌توانند به درک عمیق‌تری از مفاهیم بنیادین نظریه محاسبات، از جمله جامعیت تورینگ، ظهور پیچیدگی از سادگی، و ارتباط بین قوانین محلی و رفتارهای سراسری منجر شوند.

راهنمای اجرای تست‌ها

برای اجرای بخش‌های مختلف پروژه، دستورات زیر قابل استفاده هستند:

تست قوانین اصلی (بخش ۱-الف):

```
python test_part1a.py --finite true
```

```
python test_part1a.py --finite false
```

تست گالیدر (بخش ۱-ب):

```
python test_part1b.py --finite true
```

```
python test_part1b.py --finite false
```

تست تفنگ گالیدر (بخش ۱-پ):

```
python test_part1c.py
```

تست بارگذاری الگو از فایل (بخش ۱-ت):

```
python test_part1d.py
```

تست بهینه‌سازی کانولوشن (بخش ۱-ث):

```
python test_part1e.py --finite true
```

```
python test_part1e.py --finite false
```

نمایشگر مورچه لنگتون (بخش دوم):

```
python langton_pygame.py --size 200 --steps 10000
```

```
python langton_pygame.py --size 200 --multi-color --steps 5000
```

تست گیت‌های منطقی (بخش سوم):

`python test_logic_gates.py`

نمایشگر بازی زندگی با الگوهای مختلف:

`python pygame_gol.py`